SISTEMAS DISTRIBUIDOS · GRUPO 2 · SUSTENTACIÓN

BarberSaaS

Cómo se coordinan 31 repositorios: el workflow y el worker

Carolay Arraut · Daniel Cerquera · Juan Pablo Borrero · Carlos Leal

release v2.0.0

1 · EL PROBLEMA

Las barberías de ciudades intermedias se administran a mano

- Citas por llamada, chat o llegando: filas y clientes que se van
- Ingresos y gastos en papel, sin saber cuánto produce cada barbero
- Sin programa de fidelidad: el cliente vuelve solo por costumbre

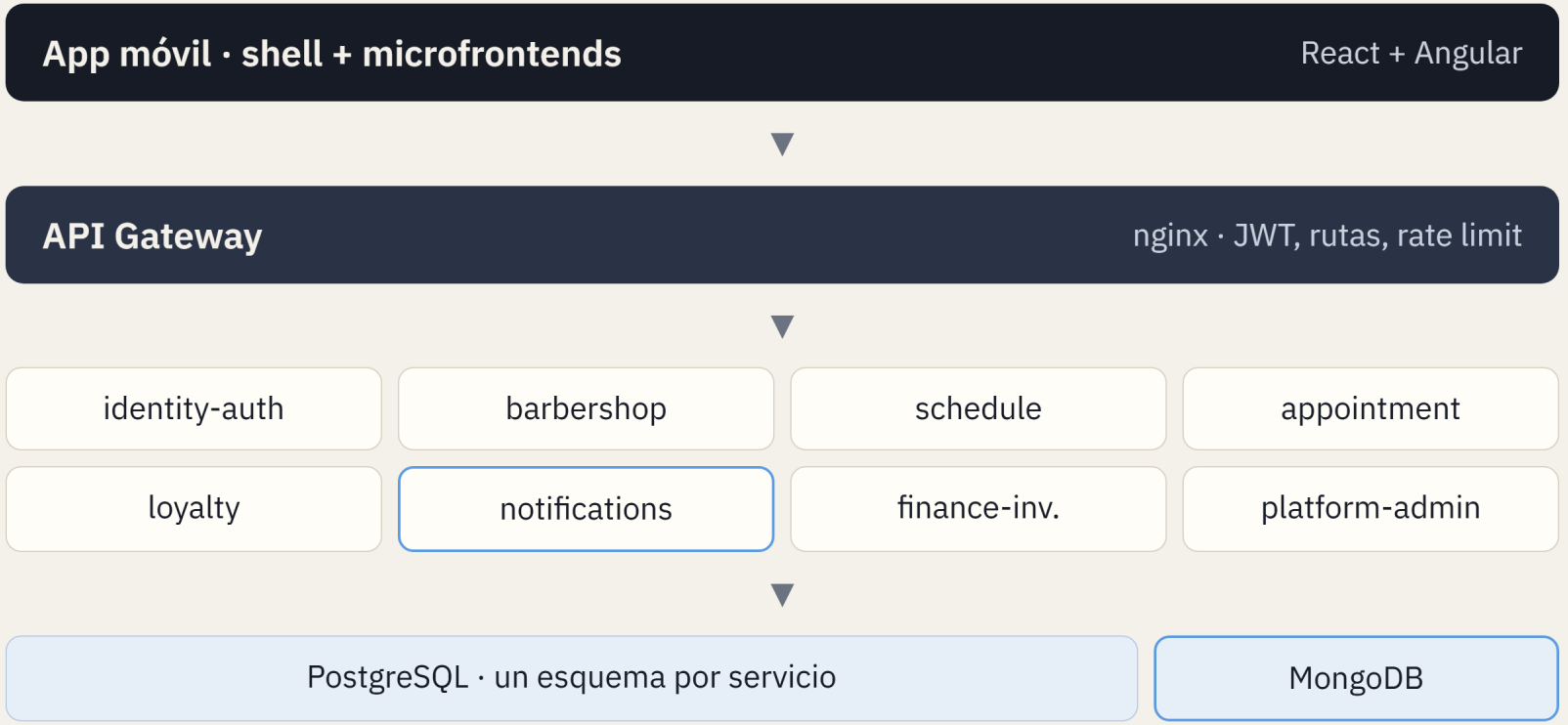
NUESTRA RESPUESTA

BarberSaaS: una plataforma para muchas barberías

Dueño, barbero, cliente y super admin: agenda, citas, fidelidad, avisos y finanzas

2 · ARQUITECTURA

8 dominios, cada uno dueño de sus datos



Workflow

Java · coordina procesos que deben ser todo o nada

Worker

Python · eventos y tareas en segundo plano

31 repos

2 lenguajes

2 motores

notifications (borde azul) es Python + MongoDB

3 · ORQUESTACIÓN DE SERVICIOS

Nadie comparte base de datos: tres piezas coordinan el sistema

API Gateway

Toda petición entra por aquí: valida el token y la lleva al servicio correcto

`síncrono · nginx`

Workflow

Coordina lo que debe pasar **todo o nada** en varios servicios, y lo deshace si falla

`síncrono · saga · Java`

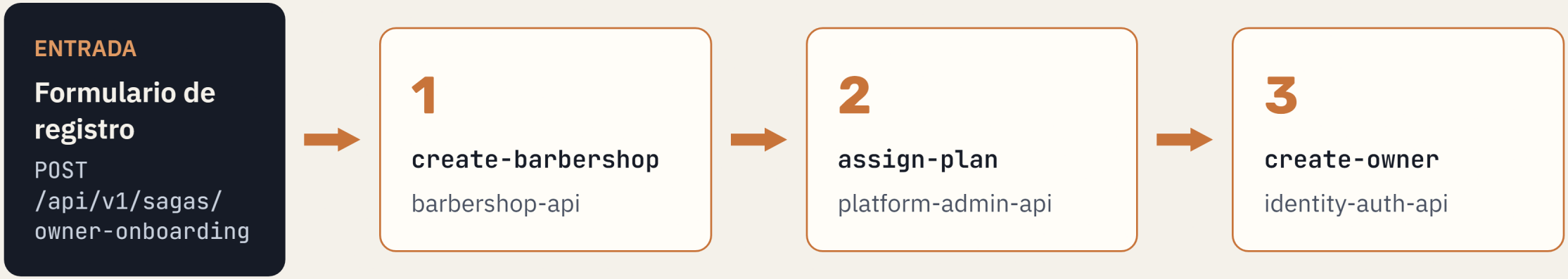
Worker

Reparte los eventos y corre las tareas diarias, **sin que el usuario espere**

`asíncrono · outbox · Python`

WORKFLOW · SAGA ORQUESTADA

Registrar un dueño toca tres servicios, en orden



Después de **cada paso** el workflow guarda el estado de la saga en su tabla: siempre se sabe en qué paso va.

WORKFLOW · COMPENSACIÓN

Si un paso falla, se deshace lo que ya se hizo

1 create-barbershop · creada

2 assign-plan · falla: plan desactivado

Compensa: borra la barbería

`COMPENSATED · PLAN_NOT_AVAILABLE`

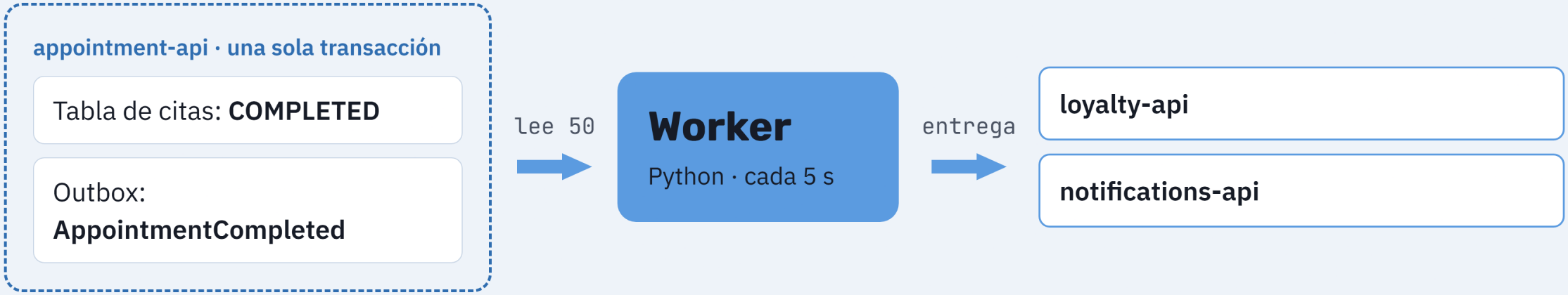
Estados de la saga

Estado	Significa
RUNNING	en curso
COMPLETED	los 3 pasos salieron bien
COMPENSATED	falló y se deshizo todo
FAILED	falló hasta deshacer: lo revisa una persona

Motivos que ve el usuario: EMAIL_ALREADY_REGISTERED, PLAN_NOT_AVAILABLE, STEP_UNAVAILABLE, INTERRUPTED

WORKER · TRANSACTIONAL OUTBOX

El evento se guarda con el dato, y el worker lo reparte



1. Pide a cada productor sus eventos pendientes (máximo 50).
2. Los entrega por POST /internal/v1/events a cada consumidor.
3. Confirma al productor: **published**, o **failed** con el motivo.

Productores: appointment, loyalty, identity-auth

4 · CAPA DE FRONT EN DOCKER

Un shell carga 8 portales, cada uno en su contenedor

Shell (front) · sesión, navegación y el único cliente HTTP :4200

identity-auth :4301 · React	**barbershop** :4302 · React	**schedule** :4303 · React	**appointment** :4304 · React
notifications :4305 · Angular	**loyalty** :4306 · Angular	**finance-inv.** :4307 · Angular	**platform-admin** :4308 · Angular

API Gateway → APIs → PostgreSQL y MongoDB :8000

Native Federation: si un portal se cae, el shell muestra **«no está disponible»** solo en esa pestaña.

DEMO EN VIVO · APAGAR Y ENCENDER UN PORTAL

Se cae Fidelidad y el resto de la app sigue funcionando

1

`docker stop bs-portal-loyalty`

Fidelidad: «no está disponible en este momento»

2

`Citas · Perfil · Avisos`

Siguen funcionando: el fallo queda aislado

3

`docker start bs-portal-loyalty`

«Intentar de nuevo» y Fidelidad vuelve

DEMO EN VIVO · WORKFLOW

Registrar un dueño, con y sin falla

Caso feliz

Registro con plan activo: los 3 pasos salen bien y el dueño entra a su barbería

`COMPLETED`

Con falla

docker stop identity-auth-api: el paso 3 falla tras 3 reintentos y la barbería se borra

`COMPENSATED · STEP_UNAVAILABLE`

```
select status, completed_steps, failed_step, failure_reason from workflow.saga
```

EN RESUMEN

Cada pieza puede caerse sin tumbar el sistema.

El gateway enruta, el workflow deshace lo incompleto, el worker no pierde eventos y el shell aísla cada portal.

¿Preguntas?